PBO 4 : Array & Arraylist

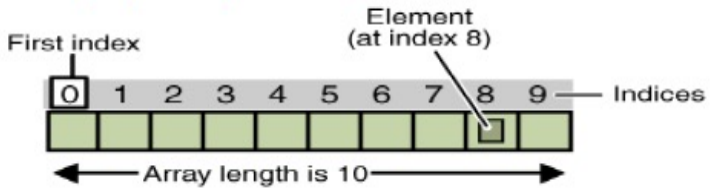


Informatics Enginering, UNRAM

2026

Array

- **Arrays** in Java are treated as **objects**.
- An array is an object that can be used to store a number of data in the same type with a fixed number of elements.



- Elements stored in an array can be primitive types (int, float, etc.) or objects (instances of classes).
- Steps to create an array :
  1. Declare the array variables
  2. Create the array object with new

```
// an array of 5 ints
int[] scores = new int[5];

// an array of Strings
String[] cities =new String[3];
```

Declaring & Creating Arrays

1. Declare the variable

```
primitiveType[] name;
ClassName[] name;

int[] number;
String[] city;
```

2. Create the array object (new)

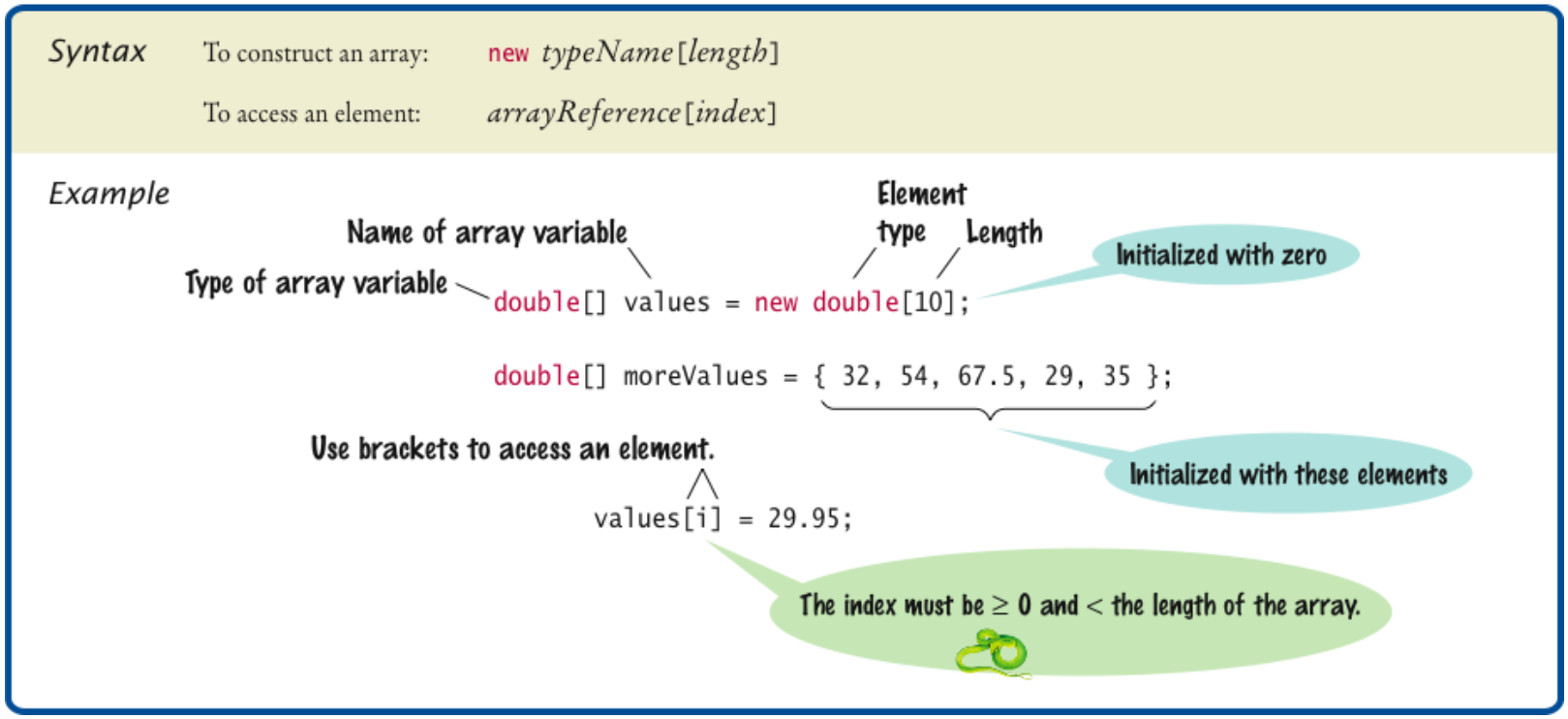
```
number = new int[7];
city   = new String[8];
```

Shorthand, declare and create in one line:

```
int[] number = new int[7];
String[] city   = new String[8];
```

NOTE The size in new int[7] is fixed at creation. Java also stores the size for you, read it with number.length (a field, not a method).

Array Declaration and Assignment



Default Initialization Values

When you create an array with new, every element gets a default value automatically.

Element type	Default value
`int, short, byte, long`	0
`double, float`	0.0
`char`	'\u0000' (null char)
`boolean`	false
`Object / String (any reference)`	null

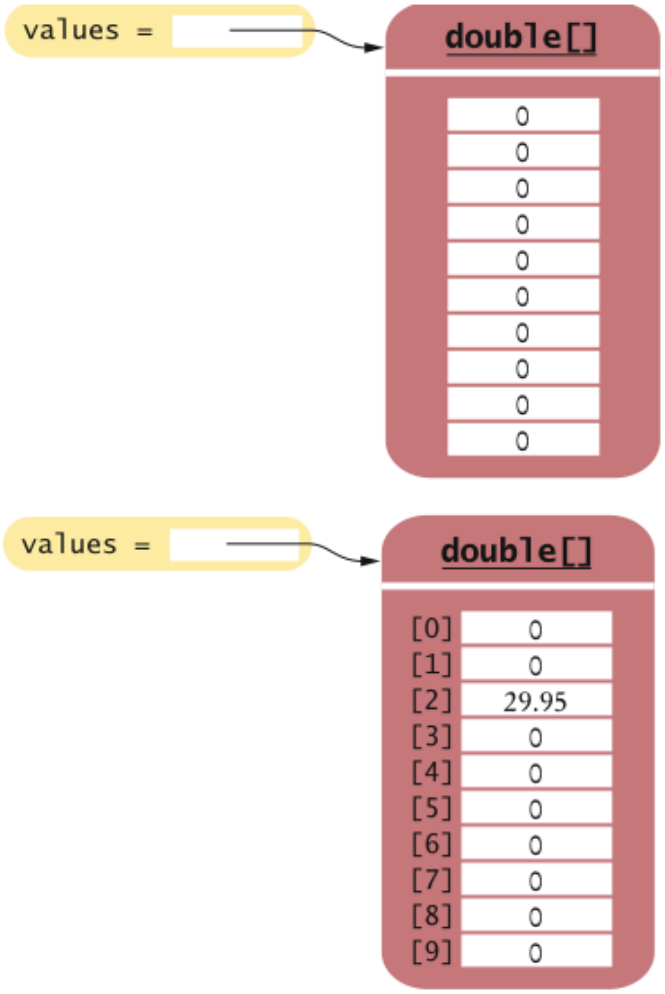
```
int[] a = new int[3];
System.out.println(a[0]);
// prints 0   (not garbage)

String[] s = new String[2];
System.out.println(s[0]);
// prints null
```

Array Declaration and Assignment

1. Array Declaration:
   double[] value = new double[10];

2. Use [ ] for access element :
   value[2] = 29.95;



Array Declaration

Table 1 Declaring Arrays

`int[] numbers = new int[10];`	An array of ten integers. All elements are initialized with zero.
`final int NUMBERS_LENGTH = 10;` `int[] numbers = new int[NUMBERS_LENGTH];`	It is a good idea to use a named constant instead of a "magic number".
`int valuesLength = in.nextInt();` `double[] values = new double[valuesLength];`	The length need not be a constant.
`int[] squares = { 0, 1, 4, 9, 16 };`	An array of five integers, with initial values.
`String[] names = new String[3];`	An array of three string references, all initially `null`.
`String[] friends = { "Emily", "Bob", "Cindy" };`	Another array of three strings.
`double[] values = new int[10]`	**Error:** You cannot initialize a `double[]` variable with an array of type `int[]`.

Accessing Elements & .length

```
public class ArrayCities {
    public static void main(String[] args) {
        String[] city = new String[3];   // create
        city[0] = "Jakarta";              // index starts at 0
        city[1] = "Surabaya";
        city[2] = "Semarang";

        System.out.println(city[0]);      // Jakarta
        System.out.println(city.length);  // 3  (a field!)
    }
}
```

Index rules

- First index is 0.
- Last valid index is length – 1.
- **city.length is a field, no ().**
- An array of size 3 has indexes 0, 1, 2.

⚠ **Direct-assignment shorthand fills values at creation: String[] city = {"Jakarta", "Surabaya", "Semarang"}; — no new needed.**

Iterating an Array with for & for-each

```
String[] city =
  {"Jakarta","Surabaya","Semarang"};

// classic for (you have the index)
for (int i = 0; i < city.length; i++) {
    System.out.println(i + ": " + city[i]);
}
```

for-each reads as: "for each String c in city".

```
// enhanced for-each (cleaner,
// when you don't need the index)
for (String c : city) {
    System.out.println(c);
}
```

NOTE Use for-each for read-only traversal — it is shorter and avoids off-by-one errors. Use the classic for when you need the index or must modify elements by position.

⚠ **for-each gives you a COPY of each value for primitives — assigning to the loop variable does not change the array. Use the index form to write back.**

ArrayIndexOutOfBoundsException

Accessing an index outside 0 .. length−1 throws a runtime exception — the program crashes if not handled.

```
int[] a = new int[3];   // valid indexes: 0, 1, 2

a[3] = 10;
// Exception in thread "main"
// java.lang.ArrayIndexOutOfBoundsException:
//   Index 3 out of bounds for length 3
```

```
// safe loop uses < length,
// never <= length
for (int i = 0; i < a.length; i++) {
    ...
}
```

NOTE The #1 cause is <= length instead of < length in a loop. This is a favourite trace/debug question — ask students to spot the off-by-one.

The Arrays Utility Class

java.util.Arrays gives ready-made helpers — no need to write loops for common tasks. Requires: import java.util.Arrays;

```
import java.util.Arrays;

int[] a = {5, 2, 8, 1};

Arrays.sort(a);                  // {1, 2, 5, 8}
System.out.println(
    Arrays.toString(a));         // [1, 2, 5, 8]

int[] b = Arrays.copyOf(a, 6);   // {1,2,5,8,0,0}
Arrays.fill(a, 7);               // {7, 7, 7, 7}
System.out.println(
    Arrays.equals(a, b));        // false
```

Most useful methods

- Arrays.toString(a) — printable form
- Arrays.sort(a) — ascending sort
- Arrays.copyOf(a,n) — resize copy
- Arrays.fill(a,v) — set all elements
- Arrays.equals(a,b) — compare contents

⚠ **Printing an array directly — System.out.println(a) — shows something like [I@1b6d3586 (the object hash), NOT the elements. Use Arrays.toString(a).**

Multidimensional Arrays

A multidimensional array is an array of arrays. Access with [row][column].

```
public class Multi {
    public static void main(String[] args) {
        String[][] name = {
            {"Mr. ", "Mrs. "},
            {"Joko", "Susi"}
        };
        System.out.println(name[0][0] + name[1][0]); // Mr. Joko
        System.out.println(name[0][1] + name[1][1]); // Mrs. Susi
    }
}
```

```
// iterate a 2D array
// with nested loops
for (int r = 0;
     r < name.length; r++) {
  for (int c = 0;
       c < name[r].length; c++){
    System.out.print(
        name[r][c] + " ");
  }
  System.out.println();
}
```

NOTE name.length is the number of rows; name[r].length is the number of columns in row r — rows may differ in length (jagged arrays).

Exercise — Countries & Capitals (2D Array)

Task

- Create class CountryCapital.
- Build a 2D array: row 0 = country, row 1 = capital.
- **Data (paired correctly):**
  - America–Washington, England–London, Japan–Tokyo, France–Paris, Indonesia–Jakarta, Iran–Tehran, Iraq–Baghdad
- **Print, e.g.: "The capital of Indonesia is Jakarta"**

```
public class CountryCapital {
 public static void main(String[] a){
  String[][] data = {
   {"America","England","Japan",
    "France","Indonesia","Iran","Iraq"},
   {"Washington","London","Tokyo",
    "Paris","Jakarta","Tehran","Baghdad"}
  };
  for (int i=0; i<data[0].length; i++){
   System.out.println("The capital of "
     + data[0][i] + " is " + data[1][i]);
  }
 }
}
```

Array Resizing

- Once an array has been created, it cannot be resized. However, it can be redeclared using the same reference.

- Example :

```
int[] myArray = new int[6];
myArray = new int[10];
```

ArrayList

Part 2

ArrayList

- `ArrayList` class manages the order of objects, which can be added to or removed as needed.
- `ArrayList` class provides many methods for sharing needs, such as adding and removing elements.
- `ArrayList` is **generic class**:
- `ArrayList<T>` collecting objects of type `T`:

```
ArrayList<String> names = new ArrayList<String>();
names.add("Emily");
names.add("Bob");
names.add("Cindy");
```

- `size` method for calculating the number of elements

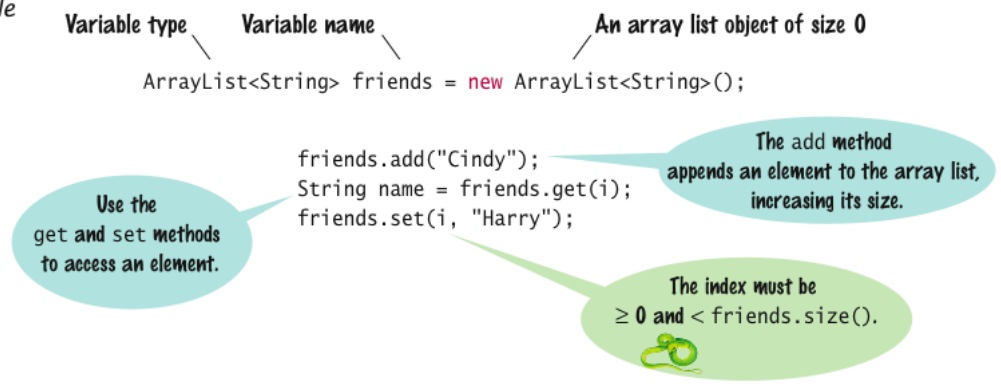
ArrayList

Syntax

To construct an array list: `new ArrayList<typeName>()`

To access an element:
arraylistReference`.get(index)`
arraylistReference`.set(index, value)`

Example



Core methods

- add(x) : append to the end
- add(i, x) : insert at index i
- get(i) — read element at i
- set(i, x) — replace element at i
- remove(i) — delete element at i
- size() — number of elements

ArrayList Holds Objects



- **A generic type parameter must be a class (reference) type, not a primitive.**
- **ArrayList<int> is illegal. Use the wrapper class: ArrayList<Integer>.**
- Autoboxing converts int ↔ Integer automatically, so it feels seamless.

```
ArrayList<Integer> nums = new ArrayList<>();  // diamond <>

nums.add(5);              // int 5 auto-boxed to Integer
nums.add(10);
int first = nums.get(0);  // Integer auto-unboxed to int

Wrapper types: int->Integer, double->Double,
               char->Character, boolean->Boolean
```

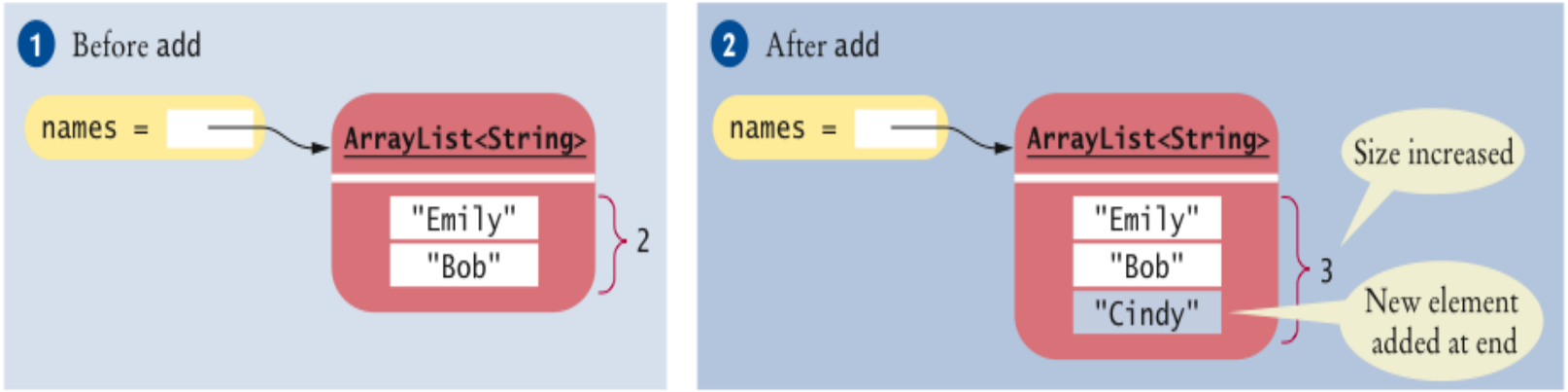
Illegal

```
ArrayList<int> x;
// compile error!
```

Add Element

- To add an element at the end of `ArrayList`, use method `add`:

```
names.add("Emily");
names.add("Bob");     1
names.add("Cindy");   2
```



Remove Element, Obtaining Element Values, Adding New Values to Elements

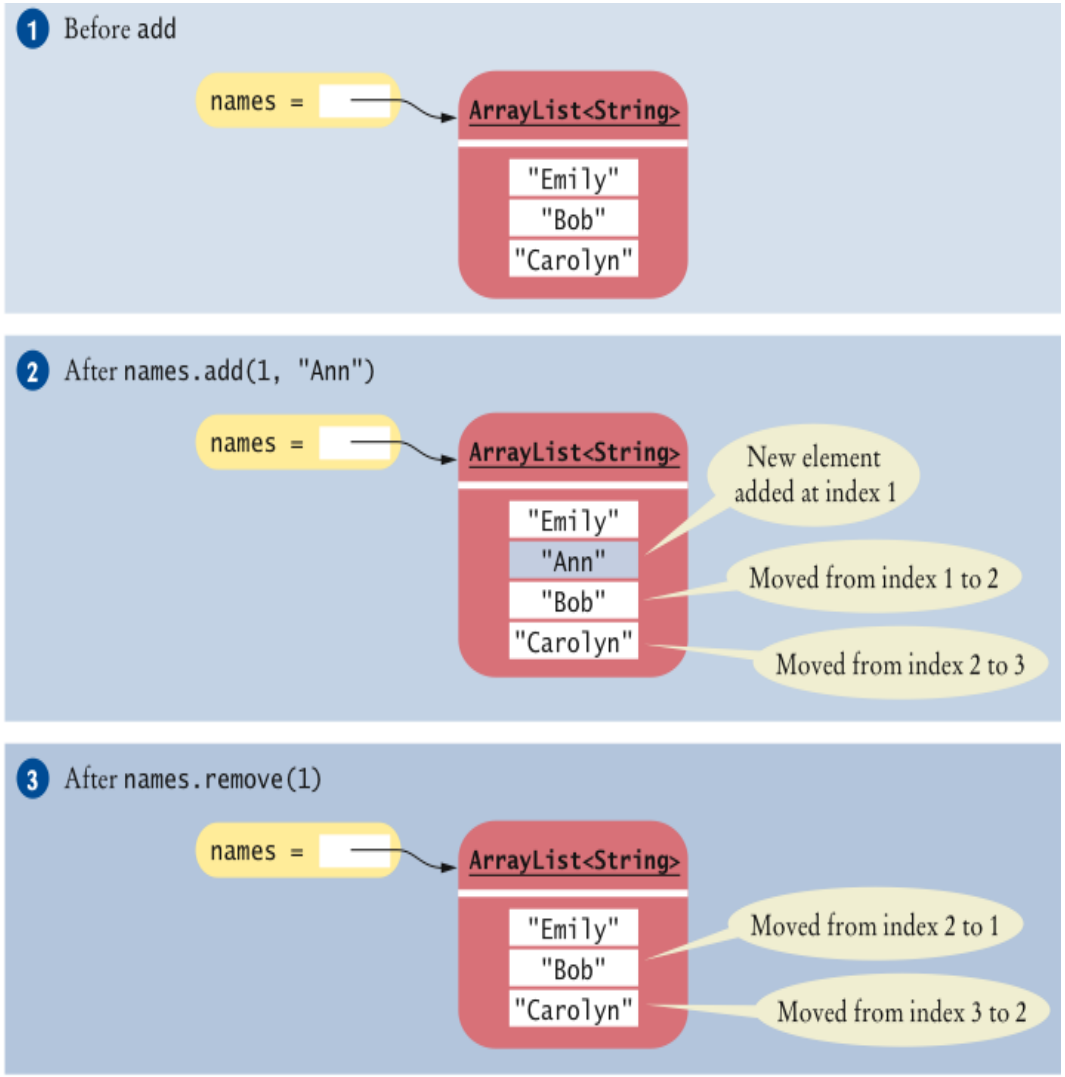
- To remove an element from an index, use the method `remove`:
  ```
  names.remove(1);
  ```

- To obtain the value of an element at an index, use the method `get`, where the index starts from 0
  ```
  String name = names.get(2);
  //dapatkan elemen ketiga dari ArrayList
  ```
- If the index is out of range, an error will occur:
  ```
  int i = names.size();
  name = names.get(i); // Error
  // legal index values are 0 ... i-1
  ```

- To add new values to an element, use the method `set`:
  ```
  names.set (2, "Carolyn");
  ```

Adding and Deleting Elements

```
names.add("Emily");
names.add("Bob");
names.add("Cindy");
names.set(2,"Carolyn");
names.add(1,"Ann");
names.remove(1);
```



ArrayList<String> names = new ArrayList<String>();	Constructs an empty array list that can hold strings
names.add("Ann"); **names.add("Cindy");**	Adds elements to the end
System.out.println(names);	Prints [Ann, Cindy]
names.add(1, "Bob");	Inserts an element at index 1. names is now [Ann, Bob, Cindy]
names.remove(0);	Removes the element at index 0. names is now [Bob, Cindy]
names.set(0, "Bill");	Replaces an element with a different value. names is now [Bill, Cindy]
String name = names.get(i);	Gets an element
String last = **names.get(names.size() - 1);**	Gets the last element

Array vs ArrayList : When to Use Which

Aspect	Array int[] / String[]	ArrayList<T>
Size	Fixed at creation	Grows / shrinks dynamically
Holds	Primitives OR objects	Objects only (use wrappers)
Access	a[i] (fast, fixed)	get(i) / set(i, x)
Size query	a.length (field)	size() (method)
Add / remove	Manual (copy to new array)	add(), remove() built in
Best when	Size known & fixed; primitives	Size changes; object collections

BankAccount.java

```
public class BankAccount {
      private double balance;
      private int accountNumber;

      public BankAccount(int accountNumber){
         balance = 0;
         this.accountNumber = accountNumber;
      }

      public void deposit(double amount){
          balance = balance + amount;
      }

      public void withdraw(double amount){
          balance = balance - amount;
      }

   public int getAccountNumber(){
       return accountNumber;
    }

   public double getBalance(){
     return balance;
   }
}
```

BankAccountArrayBeraksi.java

```
public class BankAccountArrayBeraksi{
   public static void main(String[] args) {
            ArrayList<BankAccount> accounts = new ArrayList<BankAccount>();
            accounts.add(new BankAccount(1001));
            accounts.add(new BankAccount(1015));
            accounts.add(new BankAccount(1729));
            accounts.add(1, new BankAccount(1008));
            accounts.remove(0);

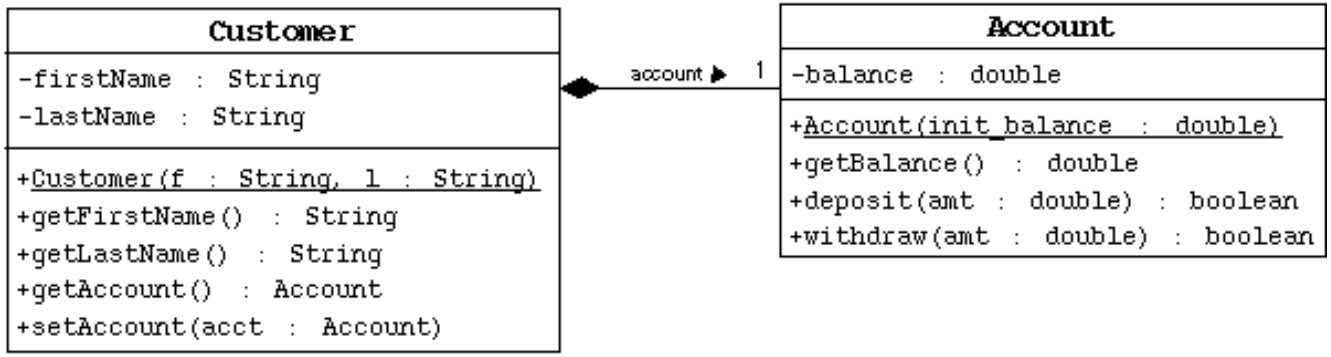
            System.out.println("Size: " + accounts.size());
            System.out.println("Expected: 3");
            BankAccount first = accounts.get(0);
            System.out.println("First account number: " + first.getAccountNumber());
            System.out.println("Expected: 1008");
            BankAccount last = accounts.get(accounts.size() - 1);
            System.out.println("Last account number: " + last.getAccountNumber());
            System.out.println("Expected: 1729");
   }
}
```

Trace the list

```
add 1001 -> [1001]
add 1015 -> [1001,1015]
add 1729 -> [1001,1015,1729]
insert 1008 @1
  -> [1001,1008,1015,1729]
remove @0
  -> [1008,1015,1729]

size = 3
get(0) = 1008
get(2) = 1729
```

Exercise : Account & Customer



- **Account.java:**
  - private double balance
  - constructor(initBalance) sets balance
  - getBalance(), deposit(amt), withdraw(amt)
- **Customer.java:**
  - private firstName, lastName, and an Account[] (or ArrayList<Account>)
  - constructor(f, l)
  - getFirstName(), getLastName()
  - setAccount(acct), getAccount()

Exercise

Create the `Account` class in the file `Account.java`

- declare one private object attribute: balance; this attribute will hold the current (or "running") balance of the bank account
- declare a public constructor that takes one parameter (init_balance); that populates the balance attribute
- declare a public method getBalance that retrieves the current balance
- declare a public method deposit that adds the amount parameter to the current balance
- declare a public method withdraw that removes the amount parameter from the current balance

Create the `Customer` class in the file `Customer.java`

- declare three private object attributes: firstName, lastName, and **account**
- declare a public constructor that takes two parameters (f and l) that populate the object attributes
- declare two public accessors for the object attributes; these methods getFirstName and getLastName simply return the appropriate attribute
- declare the setAccount method to assign the account attribute
- declare the account method to retrieve the account attribute

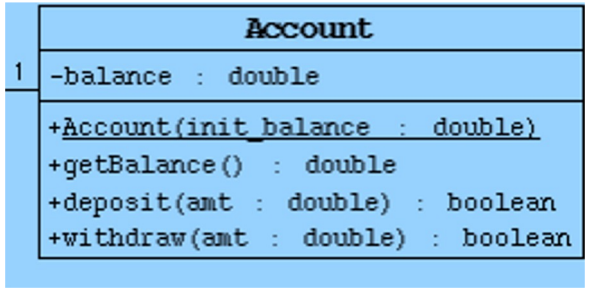
Solution

Account Class :

```
1. public class Account {
2.   protected double balance;
3.   public Account(double bal) {
4.     balance = bal;
5.   }
6.   public double getBalance() {
7.     return balance;
8.   }
9.   public boolean deposit(double amount) {
10.    if (amount>0) {
11.         balance = balance + amount;
12.        return true;
13.    } else
14.       return false;
15.}
16.  public boolean withdraw(double amount) {
17.    if ( balance >= amount ) {
18.      balance = balance - amount;
19.      return true;
20.    } else
21.      return false;
22.  }
23.}
```

Create the Account class in the file Account.java

- declare one private object attribute: balance, this attribute will hold the current (or "running") balance of the bank account
- declare a public constructor that takes one parameter (init_balance), that populates the balance attribute
- declare a public method getBalance that retrieves the current balance
- declare a public method deposit that adds the amount parameter to the current balance
- declare a public method withdraw that removes the amount parameter from the current balance



Solution

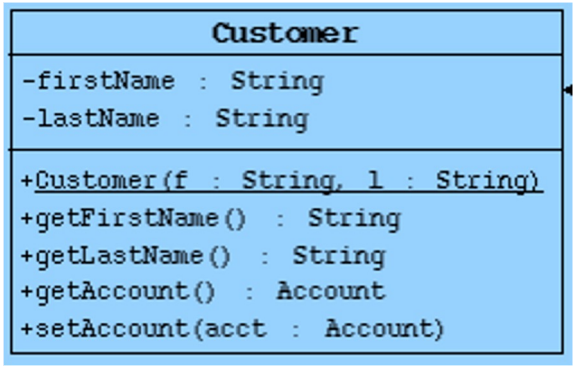
Customer Class :

```
1.    public class Customer {
2.      private String   firstName;
3.      private String   lastName;
4.      private Account[] accounts = new Account[5];
5.      private int numberOfAccounts = 0;

6.      public Customer(String f, String l) {
7.        firstName = f;
8.        lastName = l;
9.      }
10.     public String getFirstName() {
11.       return firstName;
12.     }
13.     public String getLastName() {
14.       return lastName;
15.     }
16.     public void setAccount(Account acct) {
17.       if (numberOfAccounts<5)
18.               accounts[numberOfAccounts++] = acct;
19.     }
20.     public Account getAccount(int account_index) {
21.       return accounts[account_index];
22.     }
23.     public int getNumOfAccounts() {
24.       return numberOfAccounts;
25.     }
26.   }
```

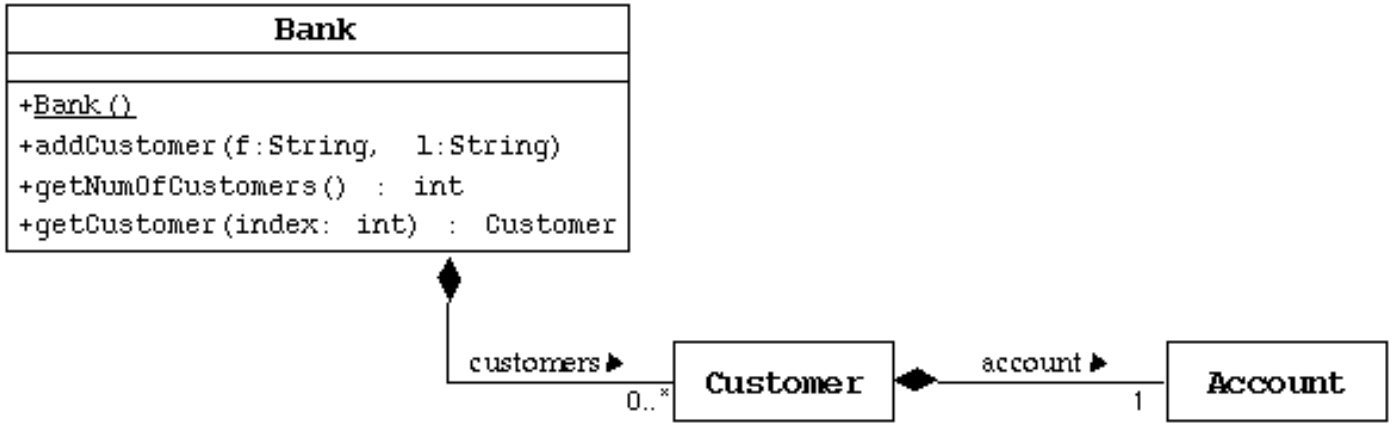
Create the Customer class in the file Customer.java

- declare three private object attributes: firstName, lastName, and account
- declare a public constructor that takes two parameters (f and l) that populate the object attributes
- declare two public accessors for the object attributes, these methods getFirstName and getLastName simply return the appropriate attribute
- declare the setAccount method to assign the account attribute
- declare the account method to retrieve the account attribute



Another Solution using ArrayList

Exercise


Bank
+Bank()
+addCustomer(f:String, l:String)
+getNumOfCustomers() : int
+getCustomer(index: int) : Customer
customers
0..*
Customer
account
1
Account



Exercise

Create the `Bank` class in the file `Bank.java`

- Add two attributes to the Bank class: customers (an array of Customer objects) and numberOfCustomers (an integer that keeps track of the next customers array index).
- Add a public constructor that initializes the customers array with some appropriate maximum size (at least bigger than 5).
- Add the addCustomer method. This method must construct a new Customer object from the parameters (first name, last name) and place it on the customers array. It must also increment the numberOfCustomers attribute.
- Add the getNumOfCustomers accessor method, which returns the numberOfCustomers attribute.
- Add the getCustomer method. This method returns the customer associated with the given index parameter.

Thank You